wxtmysql 插件深度审查报告

> 版本：0.1.12 | 类型：后端插件 | 代码量：~2,500 行
>
> 审查日期：2026-03-27

综合评分

维度	评分	简评
功能性	8/10	双池策略、连接管理完善
安全性	7/10	参数化查询，但密码明文日志可能泄露
架构设计	8/10	工厂模式 + 策略模式，设计最佳
接口设计	7/10	API 清晰但缺少类型安全
错误处理	6/10	有重试但依赖字符串匹配
可测试性	5/10	缺少接口抽象和单元测试
总评	**7/10**	全项目质量最高的插件，仍有优化空间

一、功能性分析

核心亮点：双池架构

wxtmysql 是全项目设计最出色的插件，提供了两种连接池实现：

1. **QueueLockPool**（低并发优化）
   - 使用互斥锁（`Lock`）+ FIFO 队列
   - 适合 ≤10 并发连接的场景
   - 内存占用更低
1. **SemaphorePool**（高并发优化）
   - 基于信号量的并发控制
   - 适合 10+ 并发连接的场景
   - 更好的并发吞吐量
1. 工厂自动选择

```
static WxtMysqlPool create({
  required MysqlConfig config,
  int maxConnections = 10,
  int expectedConcurrency = 5,
}) {
```

```
  if (expectedConcurrency <= 10) {
    return QueueLockPool(config: config, maxConnections: maxConnections);
  } else {
    return SemaphorePool(config: config, maxConnections: maxConnections);
  }
}
```

连接生命周期管理

功能	状态	说明
连接创建	✅	懒初始化，按需创建
连接验证	✅	`SELECT 1` 心跳检测
连接回收	✅	超时/损坏自动回收
定期维护	✅	30 秒定时器清理空闲连接
优雅关闭	✅	`dispose()` 等待所有连接归还
事务支持	✅	`withTransaction()` 自动回滚
查询重试	✅	连接丢失时自动重建 + 重试

功能缺陷

缺陷	严重度	说明
无预热机制	🟡 中	应用启动时所有连接都是冷的
无连接池指标	🟡 中	无法监控活跃/空闲/等待中的连接数
无慢查询日志	🟡 中	查询超时无感知
无读写分离	🟡 中	不支持主从读写分离

二、架构设计批判

✅ 做对的地方

1. 工厂模式：根据并发预期自动选择最优池实现
2. 策略模式：两种池实现可互换，符合 OCP
3. 连接验证：先 ping 再使用，避免拿到无效连接

4. **事务安全**：失败自动回滚 + 标记连接为损坏

缺陷

问题 1：错误检测依赖字符串匹配

```
bool _isConnectionError(dynamic error) {
  final msg = error.toString().toLowerCase();
  return msg.contains('lost connection') ||
         msg.contains('gone away') ||
         msg.contains('connection refused') ||
         msg.contains('broken pipe') ||
         msg.contains('socket') ||
         msg.contains('connection reset') ||
         msg.contains('connection closed');
}
```

批判：

- 字符串匹配极其脆弱：MySQL 不同版本、不同语言设置的错误消息不同
- 中文 locale 的 MySQL 错误信息完全不会包含这些英文关键词
- 如果 mysql1 包修改了错误消息格式，所有检测逻辑失效
- 应该检查异常类型和错误码（MySQL 错误码是标准化的）

改良方案：

```
bool _isConnectionError(dynamic error) {
  if (error is MySqlException) {
    // MySQL 标准错误码
    return const {
      2006, // CR_SERVER_GONE_ERROR
      2013, // CR_SERVER_LOST
      2003, // CR_CONN_HOST_ERROR
      1053, // ER_SERVER_SHUTDOWN
      1152, // ER_ABORTING_CONNECTION
    }.contains(error.errorNumber);
  }
  if (error is SocketException) return true;
  return false;
}
```

问题 2：重试无退避策略

```
Future<Results> query(String sql, [List<Object?>? values]) async {
  try {
    return await _executeWithConnection(sql, values);
  } catch (e) {
    if (_isConnectionError(e)) {
      // 立即重试，无任何延迟
      return await _executeWithConnection(sql, values);
    }
    rethrow;
  }
}
```

批判：

- 连接丢失后立即重试，但此时 MySQL 可能正在重启
- 只重试一次，如果 MySQL 需要 3 秒才能恢复，第二次也会失败
- 高并发场景：100 个查询同时失败 → 100 个同时重试 → 雪崩效应

改良方案：

```
Future<Results> query(String sql, [List<Object?>? values]) async {
  for (var attempt = 0; attempt < maxRetries; attempt++) {
    try {
      return await _executeWithConnection(sql, values);
    } catch (e) {
      if (!_isConnectionError(e) || attempt == maxRetries - 1) rethrow;
      final delay = Duration(
        milliseconds: min(1000 * pow(2, attempt).toInt(), 10000) +
                      Random().nextInt(1000), // 指数退避 + 随机抖动
      );
      await Future.delayed(delay);
    }
  }
  throw StateError('Unreachable');
}
```

问题 3：无断路器模式

连续 N 次连接失败后，应该暂时停止尝试（断路器打开），给 MySQL 恢复时间。当前实现会一直重试，加剧雪崩。

```
class CircuitBreaker {
  int _failures = 0;
  DateTime? _openUntil;

  bool get isOpen => _openUntil != null &&
DateTime.now().isBefore(_openUntil!);

  void recordFailure() {
    _failures++;
    if (_failures >= threshold) {
      _openUntil = DateTime.now().add(Duration(seconds: 30));
    }
  }

  void recordSuccess() {
    _failures = 0;
    _openUntil = null;
  }
}
```

问题 4：*maxWaitingRequests* 硬编码

```
final int maxWaitingRequests;
```

```
QueueLockPool({
  // ...
  this.maxWaitingRequests = 50,
});
```

等待队列上限 50 看似合理，但：

- 无法根据运行时负载动态调整
- 达到上限后直接抛异常，无优雅降级
- 没有等待超时机制（请求可能无限等待）

问题 5：维护定时器粒度不可配

```
_maintenanceTimer = Timer.periodic(
  Duration(seconds: 30),  // ← 硬编码 30 秒
  (_) => _performMaintenance(),
);
```

30 秒是硬编码的维护间隔，无法通过配置调整。低负载时 30 秒太频繁（浪费 CPU），高负载时 30 秒太慢（僵尸连接堆积）。

三、接口设计批判

问题 1：query() 返回 Results 类型外泄

```
Future<Results> query(String sql, [List<Object?>? values]);
```

`Results` 是 `mysql1` 包的类型，直接暴露在公共 API 中。如果将来迁移到 `mysql_client` 或其他 MySQL 驱动，所有调用者代码需要修改。

改良方案：

```
// 包装返回类型
class QueryResult {
  final List<Map<String, dynamic>> rows;
  final int affectedRows;
  final int? insertId;

  factory QueryResult.fromResults(Results results) { ... }
}

Future<QueryResult> query(String sql, [List<Object?>? values]);
```

问题 2：事务接口缺少隔离级别

```
Future<T> withTransaction<T>(Future<T> Function(TransactionContext) action);
```

MySQL 支持 4 种事务隔离级别，但接口中无法指定。所有事务默认使用 MySQL 全局设置的隔离级别，无法按需调整。

问题 3：MysqlConfig 过度简化

```
class MysqlConfig {
  final String host;
  final int port;
  final String user;
  final String password;
  final String db;
  final int maxConnections;
  final int timeout;
}
```

缺少：

- `charset` ：默认编码
- `timezone` ：时区设置
- `ssl` ：SSL 连接配置
- `connectTimeout` vs `readTimeout` ：读写超时区分

四、代码质量

✅ 做对的地方

1. 资源清理：`dispose()` 方法正确关闭所有连接和定时器
2. 并发安全：使用 Lock/Semaphore 保护共享状态
3. 日志分级：使用 logger 而非 print（与 authsql 形成对比）
4. 连接复用：连接归还后放回池中复用，非一次性消耗

⚠️ 需要注意

```
void _performMaintenance() async {
  // ...
  for (final conn in _idleConnections.toList()) {
    if (_isExpired(conn)) {
      _idleConnections.remove(conn);
      await conn.close();  // ← 异步操作但方法不是 async Future
    }
  }
}
```

维护方法返回类型是 `void` ，异步失败不会被捕获。应改为 `Future<void>` 并加 try-catch。

五、改良建议总结

优先级	改良项	预估工作量
P1	用 MySQL 错误码替换字符串匹配	2h
P1	实现指数退避重试策略	2h
P1	添加断路器模式	3h
P2	包装 Results → 自定义 QueryResult	3h
P2	事务接口支持隔离级别	2h
P2	添加连接池指标监控	3h
P2	维护定时器间隔可配置化	0.5h
P3	MysqlConfig 扩展（SSL, charset 等）	2h
P3	添加连接池预热机制	1h
P3	添加慢查询日志	2h